第一章 计算机系统概述

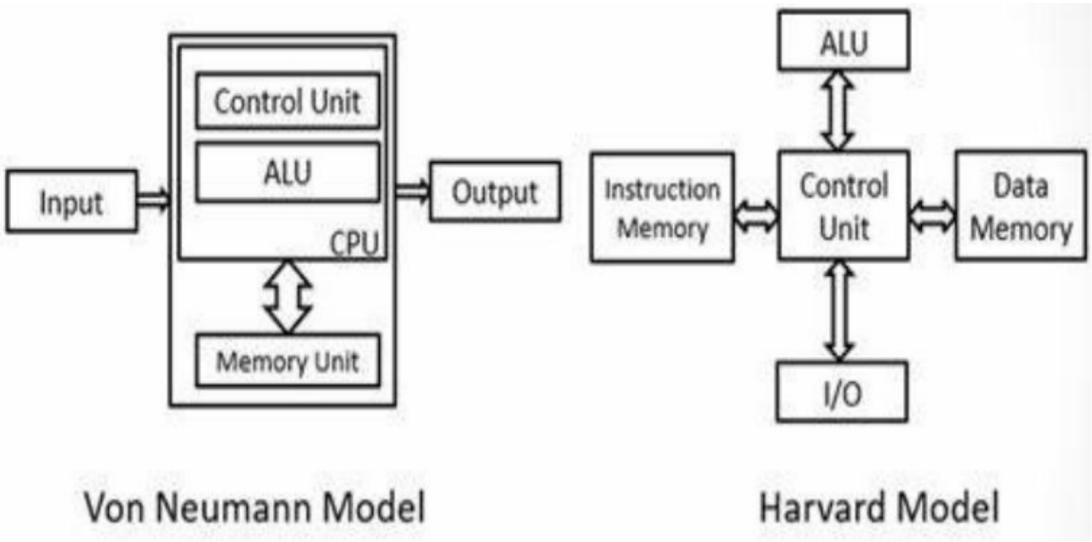
计算机体系结构中的8个伟大思想【简答题】

- 面向**摩尔定律**的设计
- 使用**抽象**简化设计
- 加速**经常性事件**
- 通过**并行**提高性能
- 通过**流水线**提高性能
- 通过**预测**提高性能
- **存储层次**
    - 速度最快、容量最小且价格最昂贵的存储器处于顶层
    - 速度最慢、容量最大且价格最便宜的存储器处于底层
- 通过**冗余**提高可靠性

冯·诺依曼计算机

- 计算机由**运算器、存储器、控制器、输入设备、输出设备**五大部件组成
- 指令和数据以同等地位存于存储器，可按地址寻访
- 指令和数据用二进制表示
- 指令由操作码和地址码组成
- **存储程序**：指令在存储器中按照顺序存放，通常是按顺序执行，特定条件下可以根据条件改变执行顺序。
- 以**运算器**为中心

哈佛架构与冯诺伊曼架构

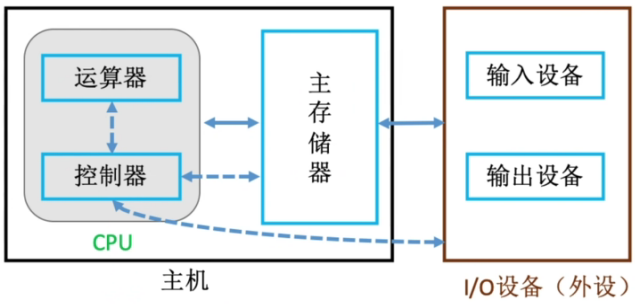


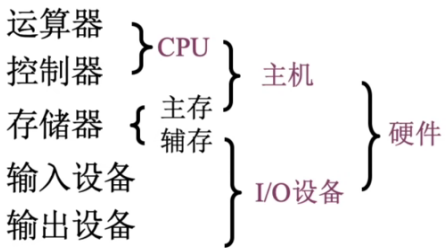
- 程序空间与数据空间是否一体?
    - 冯·诺依曼结构数据空间和程序空间不分开

    - 哈佛结构数据空间和程序空间是分开的，允许同时取指令和取操作数，从而大大提高了运算能力
- 现在的处理器
    - 外部总线上看是冯·诺依曼结构的
    - 从内部(cache)看是改进型哈佛结构。

现代计算机结构

- 以**存储器**为中心
- CPU=运算器+控制器
- 主机=运算器+控制器+主存储器





计算机性能指标【计算题】

非时间指标

- **机器字长**：CPU 一次能处理的二进制数据位数
    - 与CPU中的寄存器位数有关。一般与内部寄存器的位数相等。
    - 目前常见的是32位和64位字长。
- **总线宽度**（数据通路带宽）：数据总线一次能并行传送的最大信息位数 （各硬件部件通过数据总线传输数据）
- **主存容量**：主存包含的**存储单元总数*存储字长**

  存储字长指一个存储单元能存储的二进制代码的位数。
- **存储带宽**：单位时间内与主存交换的二进制信息量，Byte/s
- **CPU内核数**
- **存储器的容量**：
    - MAR的位数反映存储单元数量
    - MDR反映每个存储单元大小

时间指标

- **响应时间/执行时间**：计算机完成某个任务需要的总时间
- **吞吐率/带宽**：单位时间内完成的任务数量 CPU
- **主频**：CPU工作的时钟频率（单位：Hz）（1秒钟CPU可以执行的时钟周期数）
- **时钟周期**：主频的倒数 （单位：微秒、纳秒）

$$\text{CPU主频(时钟频率)} = \frac{1}{\text{CPU时钟周期}}$$

- **时钟周期数/滴答数**：时钟周期的数量
- **CPI**（指令平均时钟周期数）：执行某个程序时，**每条指令**所需的**时钟周期数**（=一段程序中所有指令的时钟周期数之和 / 指令条数）

  指令数：执行某程序所需的指令数量

  - 不同的指令，CPI不同；
  - 同一个指令，CPI也可能发生变化；

$$\text{执行一条指令耗时} = \text{CPI} \times \text{CPU时钟周期}$$

- **CPU（执行）时间**：执行某一任务在CPU上所花费的时间

$$\text{CPU执行时间} = \text{程序的指令数} \times \text{CPI} \times \text{CPU时钟周期} = \frac{\text{CPI} \times \text{程序的指令数}}{\text{CPU主频}}$$

$$\text{CPU执行时间} = \text{时钟周期长度} \times \text{CPU时钟周期数} = \frac{\text{CPU时钟周期数}}{\text{CPU主频}}$$

$$\text{程序的CPU时钟周期数} = \text{程序的指令数} \times \text{CPI}$$

- **IPC**：CPI的倒数
- **FLOPS**：每秒执行多少浮点运算
- **IPS**：每秒执行多少指令

  MIPS：每秒执行的指令数（百万）

$$\text{IPS} = \frac{\text{主频}}{\text{平均CPI}}$$

常用数量单位

- **描述频率、速率时：**

  K=Kilo=千=10^3；M=Million=百万=10^6，G=Giga=十亿=10^9，T=Tera=万亿=10^{12}
- **描述存储容量、文件大小时：**

  K=2^{10}；M=2^{20}，G=2^{30}，T=2^{40}

例：假设一台计算机主频f为1GHZ，在其上运行由2×10^5条指令组成的目标代码，程序主要由4类指令组成，他们所占的比例和各自的CPI如下表所示，求程序的CPI、MIPS和程序执行时间。

指令类型	CPI	指令比例
算术和逻辑	1	60%
Load/Store	2	18%
转移	4	12%
Cache缺失访存	8	10%

- CPI=1×60% + 2×18% + 4×12% +8×10% = 2.24
- MIPS== f ×(1/CPI)/10^6 = 1×10^9 /2.24/10^6 =446.4
- CPU时间=指令条数×CPI×1/f =(2 ×10^5)×2.24/10^9= 4.48×10^{-4}

  或者：CPU时间= 指令条数/(MIPS×10^6)= (2 ×10^5)/(446.4×10^6)

第三章 指令系统

RISC-V汇编语言

注意：

- RSIC-V实现的时候用**补码**表示
- **特殊的寄存器x0**

  x0是一个特殊的寄存器，值恒为0，且不能被改变（注意：在任意指令中，如果使用x0作为目标寄存器，**将没有任何效果**，仍然保持0不变）。例如：

  - `add x3, x0, x0`　# x3=0
  - `add x1, x2, x0`　# x1=x2
  - `add x0, x0, x0`　# nop
- 指令集的精简度

  在RISC-V中没有立即数减法，因为立即数减法可以转变为加法。

  - 例如：`addi x3, x4, -10 (in RISC)`　# f = g - 10 (in C)

  但是在RISC-V中有减法的存在，这是因为如果没有减法存在的话，则需要额外的一条指令求出负数的补码，从而实现将减法变成加法。

  RISC-V中没有NOT：因为一个数与1异或等于这个数的非，所以由于异或的存在，可以实现非运算，以达到RSIC-V的指令集最精简的目的

  - 例如：`xori  x5, x6, -1`　# x5= x6‘

  由于-1的补码为11111111_2，**所以任何数与-1异或等于这个数的非。**

  RISC-V中没有算术左移指令

RISC-V汇编指令操作对象

- 寄存器：

  32个通用寄存器，x0 ~ x31（注意：仅涉及 RV32I 的通用寄存器组）

  在 RISC-V 中，算术逻辑运算所操作的数据必须直接来自寄存器

  x0是一个特殊的寄存器，只用于全零

  每一个寄存器都有别名便于使用，实际硬件并没有任何区别

  RV32I指令集通用寄存器是32位的；RV64I指令集的通用寄存器是64位
- 内存：

  可以执行在寄存器和内存之间的数据读写操作

读写操作使用**字节（Byte）**为基本单位进行寻址

RV64可以访问最多 **2^{64}个字节的内存空间**，即**2^{61}个存储单元**

指令语法（必须记住）

类型	原始语法	示例
R	op rd, rs1, rs2	add x5, x6, x7
I	op rd, rs1, imm	addi x5, x6, -10
	（寄存器跳转）op rd, offset(rs1)	jalr x1, 100(x5)
	（load载入类） op rd, offset(rs1)	ld x5, 40(x6)
S	**（store存储类） op rs2, offset(rs1)**	sd x5, 40(x6)
B	op rs1, rs2, offset	beq, x5, x6, 100
J	op rd, offset	jal x1, 100
U	op rd, imm	**lui x10, 0x87654 # x10 = 0x87654000**

RISC-V指令表示

- RISC-V的指令都是**32位**（无论是RV32、RV64还是RV128）

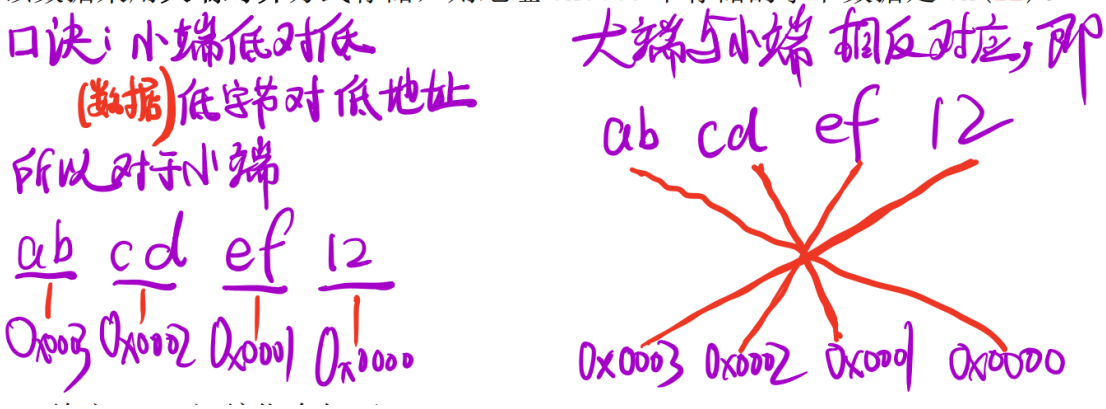
	int[31:25]	int[24:20]	int[19:15]	int[14:12]	int[11:7]	inst[6:0]
R型	func7	rs2编号	rs1编号	func3	rd编号	opcode
I型	imm[11:5]	imm[4:0]	rs1	func3	rd	opcode
S型	imm[11:5]	rs2	rs1	func3	imm[4:0]	opcode
B型	imm[12,10:5]	rs2	rs1	func3	imm[4:1,11]	opcode
J型	imm[20,10:5]	imm[4:1,11]	imm[19:12]		rd	opcode
U型	imm[31:12]				rd	opcode

建议再看一下笔记里的6条指令

考点

尾端问题：

1. 假设有一个字长的数据 0xabcdef12 采用**小端对齐**方式连续存储在地址 0x0000 至 0x0003 的内存中，则地址 0x0002 中存储的字节数据是 0x(cd)。如果该数据采用**大端对齐**方式存储，则地址 0x0003 中存储的字节数据是 0x(12)。



9、RISC-V指令系统下，假设数据0x20220319存储在地址0x4000~0x4003，则0x4002中存储的是

A 0x20

B 0x22

C 0x03

D 0x19



写汇编指令

7、int A[]首地址存储在x1中，将数组中的A[5]存储到临时寄存器x5中，下列RV语句中正确的是

A ld x5, 20(x1)

B ld x5, 40(x1)

C lw x5, 20(x1)

D lw x5, 40(x1)

本题注意 int 类型32位，所以指令 lw，偏移量 5×4=20字节

汇编要把C语言中的数据根据类型转成字节数，再写出相对数组首地址的偏移量，即偏移字节数

8、long long int A[]首地址存储在x1中，将数组中的A[6]存储到临时寄存器x5中，下列RV语句中正确的是

A ld x5, 24(x1)

B ld x5, 48(x1)

C lw x5, 24(x1)

D lw x5, 48(x1)

本题 long long int 是64位，选择ld，即load double word，对应偏移量6×8字节=48字节

汇编要把C语言中的数据根据类型转成字节数，再写出相对数组首地址的偏移量

1．给定 RV64 汇编指令 **add x18,x19,x10**，其对应的十六进制机器指令表示为 0x(00a98933 或 00A98933)。

Code	Basic
0x00a98933	add x18, x19, x10

2. 给定 RV64 汇编指令 **addi x20,x21,-10**，其对应的十六进制机器指令表示为 0x(ff6a8a13 或 FF6A8A13)。

0xff6a8a13	addi x20, x21, 0xfffffff6

3. 给定 RV64 汇编指令 **ld x10,-100(x8)**，其对应的十六进制机器指令表示为 0x(f9c43503 或 F9C43503)。

0xf9c43503	ld x10, 0xffffff9c(x8)

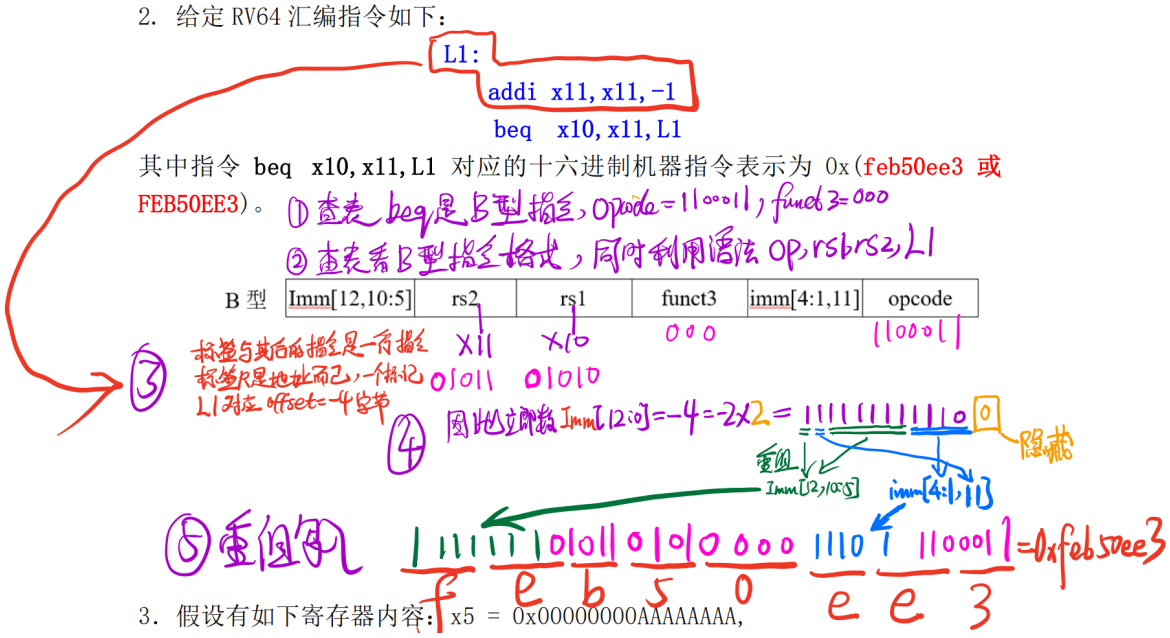
4. 给定 RV64 汇编指令 **sd x11,-16(x9)**，其对应的十六进制机器指令表示为 0x(feb4b823 或 FEB4B823)。

0xfeb4b823	sd x11, 0xfffffff0(x9)

2. 给定 RV64 汇编指令如下：

```
L1:
    addi  x11, x11, -1
    beq   x10, x11, L1
```

其中指令 beq x10, x11, L1 对应的十六进制机器指令表示为 0x(feb50ee3 或 FEB50EE3)。



3．假设有如下寄存器内容：x5 = 0x00000000AAAAAAAA,

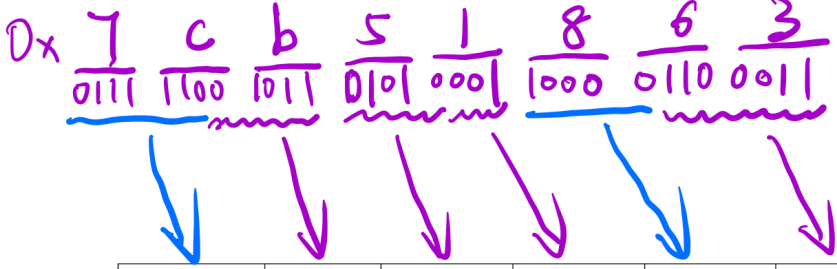
跳转了1个指令--1个字--4个字节---（-4）/2 = -2

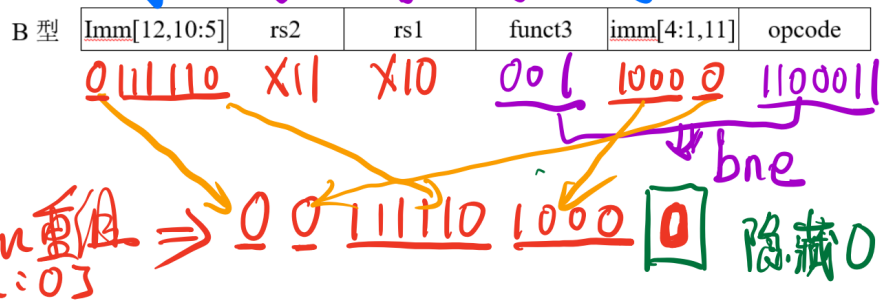
$-2_{补}$=111111111110

1111111 01011 01010 000 11101 1100011

指令表示--汇编语言

1）给定 RV 机器码 0x7cb51863，写出汇编指令 **bne x10，x11，2000**

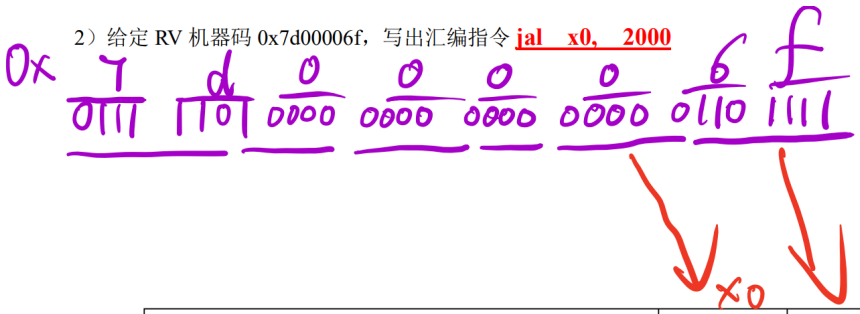


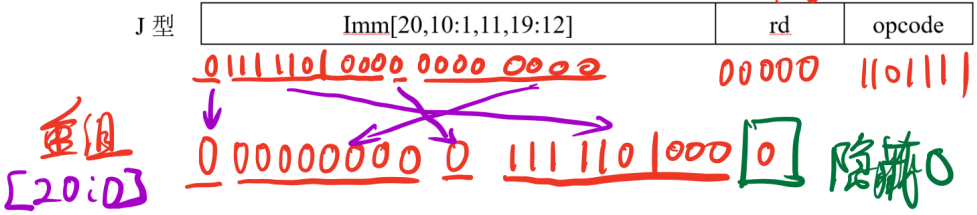


offset=0011111010000=2000

B型指令 op rs1, rs2, offset ∴ bne x10, x11, 2000

2）给定 RV 机器码 0x7d00006f，写出汇编指令 **jal x0，2000**

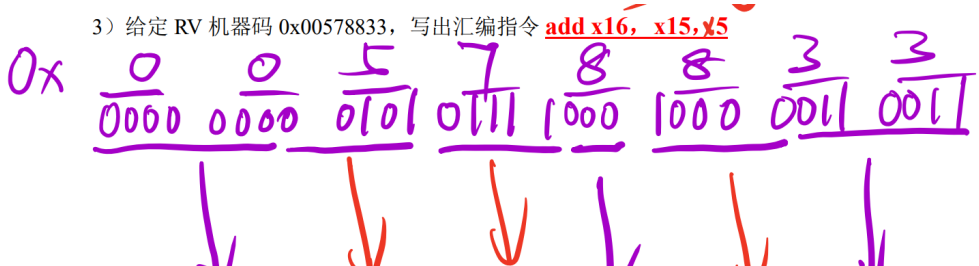


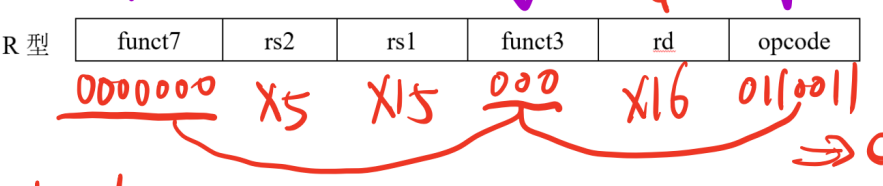


offset=00000000011110100000=2000

⇒ jal x0, 2000

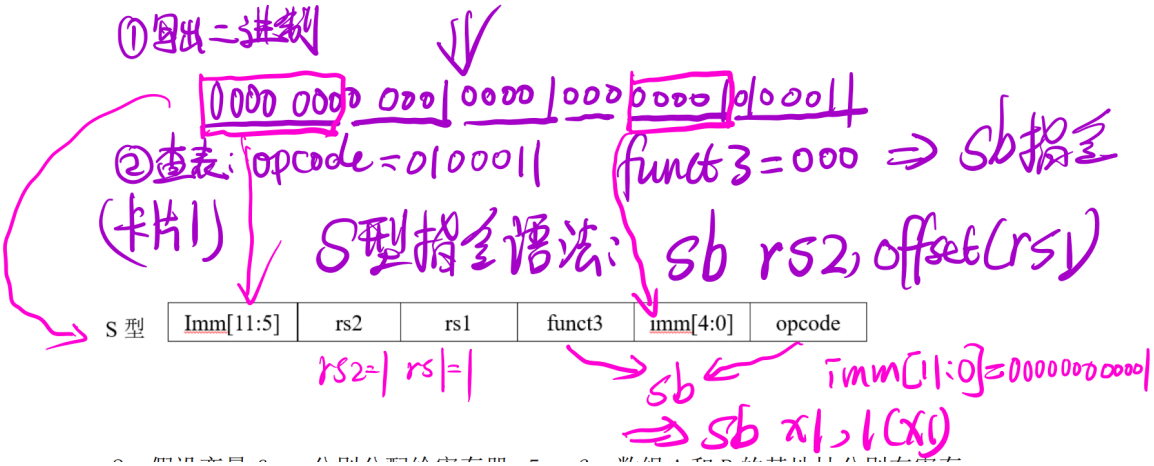
3）给定 RV 机器码 0x00578833，写出汇编指令 **add x16，x15，x5**





清除：add rd, x1, x2 ⇒ add x16, x15, x5

1. 给定十六进制机器指令 0x001080A3，查表写出其对应的 RV64 汇编指令。



判断程序结果：

4）执行下列指令序列后，寄存器x10的值是 [填空1] ？（填写十进制的值）。

```
addi x10, x0, 4
bge x10, x0, L1
    srli x10, x10, 1
    j Exit
L1:
    slli x10, x10, 2
Exit:
```

① x10=4
② x10>=x0跳转L1
③ x10=x10×4=16
④ 结束

执行下列指令序列后，寄存器x10的值是多少？

```
addi x10, x0, 4
bge x10, x0, L1
    srli x10, x10, 1
    j Exit
L1:
    slli x10, x10, 2
Exit:
```

解析：
1：执行后x10=4
2：判断x10的值是否大于等于0？如果大于等于跳转L1，否则，顺序执行。
这里x10=4大于0，跳转L1执行。
L1：x10寄存器逻辑左移2位，执行后x10=16
Exit：退出程序
综上，执行下列指令后，x10的值为16

5） 执行下列指令序列后，寄存器x10的值是 [填空1] ？（填写十进制的值）。

```
addi x11, x0, 11
mv x5, x0
mv x10, x0
Loop:
    bge x5, x11, Done
    add x10, x10, x5
    addi x5, x5, 1
    j Loop
Done:
```

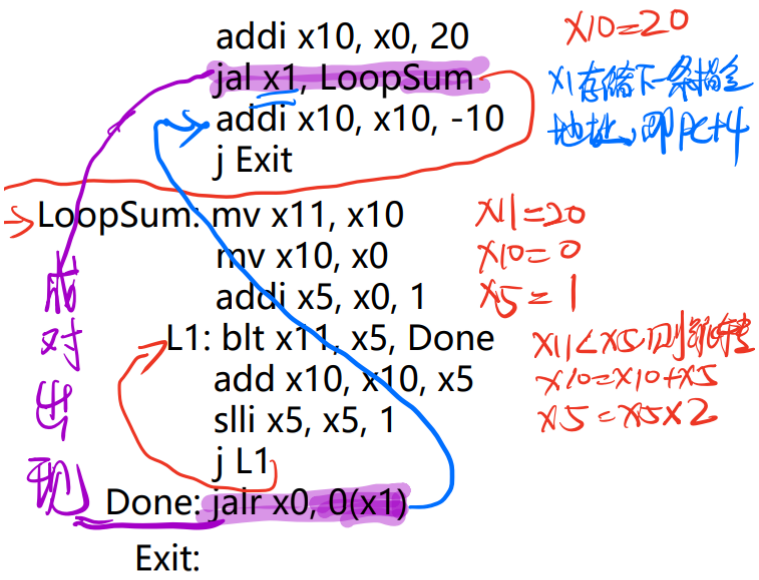
执行下列指令序列后，寄存器x10的值是多少？

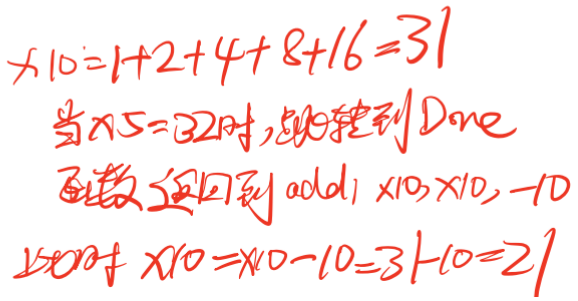
```
addi x11, x0, 11
mv x5, x0
mv x10, x0
Loop:
    bge x5, x11,
    Done
    add x10, x10, x5
    addi x5, x5, 1
    j Loop
Done:
```

解析：
整个程序的流程相当于使用for循环求1+2+3+...+10的值，结果存入x10寄存器。
1：执行后x11=11
2、3：使用mv指令将x5和x10寄存器清0。
Loop:
判断寄存器x5的值是否大于等于x11的值(11)如果大于则跳转Done，程序执行完成。
否则，累加寄存器x10的值，x10 = x10 + x5。然后将寄存器x5的值+1，无条件跳转到循环起始位置，开始下一轮循环。
最终x10 = 1+2+3+...+10 = 55

6）执行下列指令序列后，
寄存器x10的值是 [填空1]?
（填写十进制的值）。

```
        addi x10, x0, 20
        jal x1, LoopSum
        addi x10, x10, -10
        j Exit
LoopSum: mv x11, x10
        mv x10, x0
        addi x5, x0, 1
    L1: blt x11, x5, Done
        add x10, x10, x5
        slli x5, x5, 1
        j L1
Done: jalr x0, 0(x1)
Exit:
```

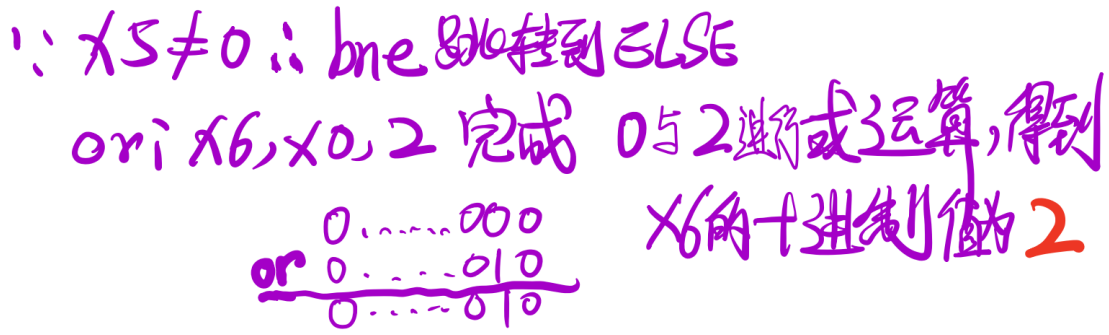




4．假设 x5 保存 0x0000000000101000。以下指令完成后 x6 的十进制值是(?)。

```
        bne x5, x0, ELSE
        jal x0, DONE
ELSE:   ori x6, x0, 2
DONE:
```

2



判断范围：

4、RISC-V 中条件分支的字节地址范围是多少？（ **D** ）

A、地址在 0 到 4K-1 之间

B、地址在 0 到 8K-1 之间

C、分支前后地址范围各大约 2K

D、分支前后地址范围各大约 4K

点评：B 型指令含隐藏位共 13 位，前后各大约 2^{12} 字节=即前后各大约 4K 字节

精确来说：-2^{12}～$2^{12}-2$ 字节(隐藏位始终是 0，不存在奇数)，-4096~4094 字节

5、RISC-V 中 jal 的字节地址范围是多少？（ **D** ）

A、地址在 0 到 512K-1 之间

B、地址在 0 到 1M-1 之间

C、分支前后地址范围各大约 512K

D、分支前后地址范围各大约 1M

点评：J 型指令含隐藏位共 21 位，前后各大约 2^{20} 字节=即前后各大约 1M 字节

精确来说：-2^{20}～$2^{20}-2$ 字节 (隐藏位始终是 0，不存在奇数)，-1M~1M-2 字节